



Chapter 1

Project Zero: A Template for Other Projects

This is a book of projects. To make each project a good portfolio piece, we'll treat each project as an enterprise software product. You can build something that could be posted to a company's (or organization's) internal repository.

For this book, we'll define some standards that will apply to all of these projects. The standards will identify deliverables as a combination of files, modules, applications, notebooks, and documentation files. While each enterprise is unique, the standards described here are consistent with my experience as a consultant with a variety of enterprises.

We want to draw an informal boundary to avoid some of the steps required to post to the PyPI website. Our emphasis is on a product with test cases and enough documentation to explain what it does. We don't want to go all the way to creating a project in PyPI. This allows us to avoid the complications of a build system and the associated `pyproject.toml` file.

These projects are not intended to produce generic, reusable modules. They're applications specific to a problem domain and a dataset. While these are specific solutions, we don't want to discourage anyone who feels motivated to generalize a project into something generic and reusable.

This chapter will show the general outline of each project. Then we'll look at the set of deliverables. This chapter ends with project zero – an initial project that will serve as a template for others. We'll cover the following topics:

- An overview of the software quality principles that we'll try to emphasize.
- A suggested approach to completing the project as a sequence of project sprints.
- A general overview of the list of deliverables for each project.
- Some suggested tools. These aren't required, and some readers may have other choices.
- A sample project to act as a template for subsequent projects.

We'll start with an overview of some characteristics of high-quality software. The idea is to establish some standards for the deliverables of each project.

1.1 On quality

It helps to have a clear definition of expectations. For these expectations, we'll rely on the ISO 25010 standard to define quality goals for each project. For more details, see

<https://iso25000.com/index.php/en/iso-25000-standards/iso-25010>.

The ISO/IEC 25010:2011 standard describes **Systems and software Quality Requirements and Evaluation (SQuaRE)**. This standard provides eight characteristics of software. These characteristics are as follows:

- **Functional suitability.** Does it do what we need? It is complete, correct, and appropriate for the user's expressed (and implied) needs? This is the focus of each project's description.
- **Performance efficiency.** Does it work quickly? Does it use the minimum resources? Does it have enough capacity to meet the user's needs? We won't address this deeply in this book. We'll talk about

writing performance tests and ways to address performance concerns.

- **Compatibility.** Does it co-exist with other software? Does it properly interoperate with other applications? To an extent, Python can help assure an application interoperates politely with other applications. We'll emphasize this compatibility issue in our choices of file formats and communication protocols.
- **Usability.** There are a number of sub-characteristics that help us understand usability. Many of the projects in this book focus on the **command-line interface (CLI)** to assure a bare minimum of learnability, operability, error protection, and accessibility. A few projects will include a web services API, and others will make use of the GUI interface of JupyterLab to provide interactive processing.
- **Reliability.** Is it available when the users want it? Can we detect and repair problems? We need to make sure we have all of the parts and pieces so we can use the software. We also need to make sure we have a complete set of tests to confirm that it will work.
- **Security.** As with usability, this is a deep topic. We'll address some aspects of security in one of the projects. The remaining projects will use a CLI permitting us to rely on the operating system's security model.
- **Maintainability.** Can we diagnose problems? Can we extend it? We'll look at documentation and test cases as essential for maintainability. We'll also leverage a few additional project files to make sure our project can be downloaded and extended by others.
- **Portability.** Can we move to a new Python version? New hardware? This is very important. The Python ecosystem is rapidly evolving. Since all of the libraries and packages are in a constant state of change, we need to be able to define precisely what packages our project depends on, and confirm that it works with a new candidate set of packages.

Two of these characteristics (Compatibility and Portability) are features of Python. A wise choice of interfaces assures that these characteristics

are met. These are sometimes described as architectural decisions since they influence how multiple applications work together.

For Security, we will rely on the operating system. Similarly, for Usability, we'll limit ourselves to CLI applications, relying on long-standing design principles.

The idea of Performance is something we won't emphasize here. We will point out places where large data sets will require some careful design. The choice of data structure and algorithm is a separate subject area. Our objective in this book is to expose you to projects that can provide the stimulus for a deeper study of performance issues.

Three of these quality characteristics — Functional suitability, Reliability, and Maintainability — are the real focus of these projects. These seem to be essential elements of good software design. These are the places where you can demonstrate your Python programming skills.

Another view is available from **The Twelve-Factor App** (<https://12factor.net>). This is narrowly focused on web applications. The concepts provide deeper insights and more concrete technical guidance into the quality characteristics shown above:

1. Codebase. "One codebase tracked in revision control, many deploys." We'll use **Git** and **GitHub** or perhaps one of the other version managers supported by **sourceforge**.
2. Dependencies. "Explicitly declare and isolate dependencies." Traditionally, a Python `requirements.txt` file was used for this. In this book, we'll move forward to using a `pyproject.toml` file.
3. Config. "Store config in the environment." We won't emphasize this, but Python offers numerous ways to handle configuration files.
4. Backing services. "Treat backing services as attached resources." We touch on this in a few places. How storage, messages, mail, or caching work isn't something we'll examine deeply.
5. Build, release, run. "Strictly separate build and run stages." For command-line applications, this means we should deploy the application

into a "production" environment to use the high-value data and produce the results that the enterprise needs. We want to avoid running things in our desktop development environment.

6. Processes. "Execute the app as one or more stateless processes." CLI applications tend to be structured this way without making any additional effort.
7. Port binding. "Export services via port binding." We won't emphasize this; it's very specific to web services.
8. Concurrency. "Scale out via the process model." This is a subject for the interested reader who wants to process very large data sets. We won't emphasize it in the main text. We will suggest some of these topics in the "Extras" section of some chapters.
9. Disposability. "Maximize robustness with fast startup and graceful shutdown." CLI applications tend to be structured this way, also.
10. Dev/prod parity. "Keep development, staging, and production as similar as possible." While we won't emphasize this deeply, our intent with CLI applications is to expose the distinctions between development and production with command-line arguments, shell environment variables, and configuration files.
11. Logs. "Treat logs as event streams." We will suggest applications write logs, but we won't provide more detailed guidance in this book.
12. Admin processes. "Run admin/management tasks as one-off processes." A few of the projects will require some additional administrative programming. These will be built as deliverable CLI applications, complete with an acceptance test suite.

Our objective is to provide project descriptions and lists of deliverables that try to conform to these quality standards. As we noted earlier, each enterprise is unique, and some organizations will fall short of these standards, while some will exceed them.

1.1.1 More Reading on Quality

In addition to the ISO standard, the IEEE 1061 standard also covers software quality. While it has been inactive since 2020, it contains some

good ideas. The standard is focused on quality *metrics*, which dives deeply into the idea of analyzing software for quality factors.

It can also help to read https://en.wikipedia.org/wiki/ISO/IEC_9126 for some background on the origins of the ISO standard.

When doing more reading on this topic, it can help to recognize the following three terms:

- **Factors** are an external view of the software. They reflect the user's understanding. Some of the underlying quality characteristics are not directly visible to users. Maintainability, for example, may appear to users as a reliability or usability problem because the software is difficult to repair or extend.
- **Criteria** come from an internal view of the software. Quality criteria are the focus of the project's deliverables. Our project code should reflect the eight quality characteristics listed above.
- **Metrics** are how we can control the factors that are seen by the user. We won't emphasize quality metrics. In some cases, tools like **pylint** provide tangible measurements of static code quality. This isn't a comprehensive tool for software quality in general, but it provides an easy starting point for a few key metrics related to complexity and maintainability.

Given these standards for high-quality software, we can turn our attention to the sequence of steps for building these files. We'll suggest a sequence of stages you can follow.

1.2 Suggested project sprints

We hesitate to provide a detailed step-by-step process for building software. For more experienced developers, our sequence of steps may not match their current practices. For less experienced developers, the suggested process can help by providing a rational order in which the deliverables can be built.

There was a time when a "statement of work" with a detailed list of specific tasks was a central part of a software development effort. This was often part of a "waterfall" methodology where requirements flowed to analysts who wrote specifications that flowed down to designers who wrote high-level designs that flowed down to coders. This wasn't a great way to build software, and has been largely supplanted by Agile methods. For more information on Agility, see <https://agilemanifesto.org>.

The Agile approach lets us examine a project both as a series of steps to be completed, as well as a collection of deliverables that need to be created. We'll describe the steps first, avoiding too much emphasis on details. We'll revisit the deliverables, and in those sections, dive a little more deeply into what the final product needs to be.

The suggested approach follows the "Agile Unified Process" (<https://www.methodsandtools.com/archive/archive.php?id=21>), which has four general phases. We'll subdivide one of the phases to distinguish two important kinds of deliverables.

We suggest tackling each project in the following five phases:

1. Inception. Ready the tools. Organize the project directory and virtual environment.
2. Elaboration, part 1: Define done. This is implemented as acceptance test cases.
3. Elaboration, part 2: Define components and some tests. This is implemented as unit test cases for components that need to be built.
4. Construction. Build the software.
5. Transition. Final cleanup: make sure all tests pass and the documentation is readable.

These efforts don't proceed in a simple linear fashion. It's often necessary to iterate between elaboration and construction to create features separately.

It often works as shown in [*Figure 1.1*](#).

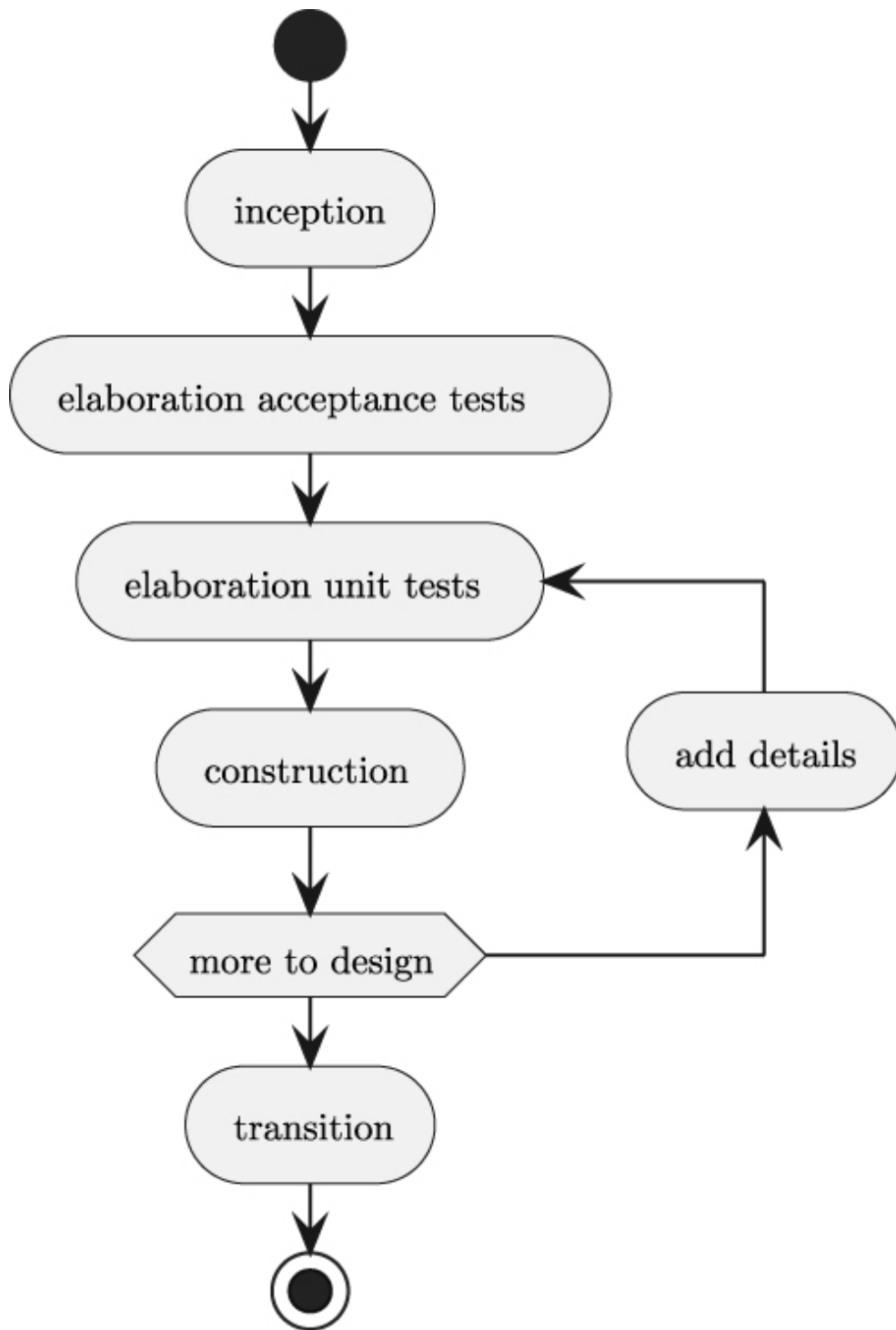


Figure 1.1: Development Phases and Cycles

This figure provides a very coarse overview of the kinds of activities we'll discuss below. The important concept is iterating between the elaboration and construction phases. It's difficult to fully design a project before constructing all of the code. It's easier to design a little, construct a little, and refactor as needed.

For a complex project, there may be a series of transitions to production. Often a “minimally viable product” will be created to demonstrate some of the concepts. This will be followed by products with more features or features better focused on the user. Ideally, it will have both kinds of enhancements: more features and a better focus on the user’s needs.

We’ll look at each of these four phases in a little more detail, starting with the inception phases.

1.2.1 Inception

Start the inception phase by creating the parent directory for the project, then some commonly-used sub-directories (`docs` , `notebooks` , `src` , `tests`). There will be some top-level files (`README.md` , `pyproject.toml` , and `tox.ini`). The list of expected directories and files is described in more detail in [List of deliverables](#), later in this chapter. We’ll look at the contents of each of these files and directories in the [Deliverables](#) section.

It helps to capture any initial ideas in the `README.md` file. Later, this will be refactored into more formal documentation. Initially, it’s the perfect place to keep notes and reminders.

Build a fresh, new virtual environment for the project. Each project should have its own virtual environment. Environments are essentially free: it’s best to build them to reflect any unique aspects of each project.

Here’s a **conda** command that can be used to build an environment.

```
% conda create -n project0 --channel=conda-forge python=3.10
```

An important part of inception is to start the documentation for the project. This can be done using the Sphinx tool.

While Sphinx is available from the **Conda Forge**, this version lags behind the version available from the **PyPI** repository. Because of this lag, it's best to install Sphinx using **PIP**:

```
% python -m pip install sphinx
```

After installing Sphinx, it helps to initialize and publish the documentation for the project. Starting this permits publishing and sharing the design ideas as the work progresses. In the `docs` directory, do the following steps:

1. Run the `sphinx-quickstart` command to populate the documentation. See <https://www.sphinx-doc.org/en/master/usage/quickstart.html#setting-up-the-documentation-sources>.
2. Update the `index.rst` table of contents (TOC) with two entries: “overview” and “API”. These are sections that will be in separate files.
3. Write an `overview.rst` document with the definition of done: what will be accomplished. This should cover the core “Who-What-When-Where-Why” of the project.
4. Put a title in the API document, and a `.. todo::` note to yourself. You'll add to this document as you add modules to your project.
5. During Elaboration, you'll update the `index.rst` to add sections for architecture and design decisions.
6. During Construction, as you create code, you'll add to the API section.
7. During Transition, you'll add to the `index.rst` with some “How” sections: How to test it, and how to use it.

With this as the starting point, the `make html` command will build a documentation set in HTML. This can be shared with stakeholders to assure there's a clear, common understanding of the project.

With a skeleton directory and some initial places to record ideas and decisions, it makes sense to start elaborating on the initial goal to and decide what will be built, and how it will work.

1.2.2 Elaboration, part 1: define done

It helps to have a clear definition of "done." This guides the construction effort toward a well-defined goal. It helps to have the definition of done written out as a formal, automated test suite. For this, the Gherkin language is helpful. The **behave** tool can execute the Gherkin feature to evaluate the application software. An alternative to Gherkin is using the **pytest** tool with the **pytest-bdd** plug-in to run the acceptance tests.

The two big advantages of Gherkin are the ability to structure the feature descriptions into scenarios and write the descriptions in English (or any other natural language). Framing the expected behavior into discrete operating scenarios forces us to think clearly about how the application or module is used. Writing in English (or other natural languages) makes it easier to share definitions with other people to confirm our understanding. It also helps to keep the definition of done focused on the problem domain without devolving into technical considerations and programming.

Each scenario can have three steps: Given, When, and Then. The Given step defines a context. The When step defines an action or a request of the software. The Then step defines the expected results. These step definitions can be as complex as needed, often involving multiple clauses joined with **And**. Examples can be provided in tables to avoid copying and pasting a scenario with a different set of values. A separate module provides Python implementations for the English-language step text.

See <https://behave.readthedocs.io/en/stable/gherkin.html#gherkin-feature-testing-language> for numerous examples of scenarios written in Gherkin.

Start this part of elaboration by creating a `tests/features/project.feature` file based on the overview description. Don't use a boring name like `project`. A complex project may have multiple features, so the feature file names should reflect the features.

To use **pytest**, write one (or more) acceptance test scripts in the `tests` directory.

The features are supported by **steps**. These steps are in modules in the `tests/steps` directory. A `tests/steps/hw_cli.py` module provides the necessary Python definitions for the steps in the feature file. The names of the modules don't matter; we suggest something like `hw_cli` because it implements the steps for a hello-world command-line interface.

The underlying mechanism is used by the **Behave** tool are function decorators. These match text from the feature file to define the function that implements that step. These can have wildcard-matching to permit flexibility in wording. The decorator can also parse out parameter values from the text.

A `tests/environment.py` file is required, but it can be empty for simple tests. This file provides a testing context, and is where some functions used by the **Behave** tool to control test setup and teardown are defined.

As soon as scenarios have been written, it makes sense to run the **Behave** tool to see the acceptance test fail. Initially, this lets you debug the step definitions.

For this application, the steps must properly execute the application program and capture the output file. Because the application doesn't exist yet, a test failure at this point is expected.

The feature files with the application scenarios are a working definition of done. When the test suite runs, it will show whether or not the software works. Starting with features that fail to work means the rest of the construction phase will be debugging the failures and fixing the software until the application passes the acceptance test suite.

In ***Project 0 – Hello World with test cases*** we'll look at an example of a Gherkin-language feature, the matching step definitions, and a `tox.ini` to run the test suite.

1.2.3 Elaboration, part 2: define components and tests

The acceptance test suite is often relatively "coarse" – the tests exercise the application as a whole, and avoid internal error conditions or subtle edge cases. The acceptance test suite rarely exercises all of the individual software components. Because of this, it can be difficult to debug problems in complex applications without detailed unit tests for each unit — each package, module, class, and function.

After writing the general acceptance test suite, it helps to do two things. First, start writing some skeleton code that's likely to solve the problem. The class or function will contain a docstring explaining the idea. Optionally, it can have a body of the `pass` statement. After writing this skeleton, the second step is to expand on the docstring ideas by writing unit tests for the components.

Let's assume we've written a scenario with a step that will execute an application named `src/hello_world.py`. We can create this file and include a skeleton class definition like this:

```
class Greeting:
    """
    Created with a greeting text.
    Writes the text to stdout.
```

```
.. todo:: Finish this
"""
pass
```

This example shows a class with a design idea. This needs to be expanded with a clear statement of expected behaviors. Those expectations should take the form of unit tests for this class.

Once some skeletons and tests are written, the **pytest** tool can be used to execute those tests.

The unit tests will likely fail because the skeleton code is incomplete or doesn't work. In the cases where tests are complete, but classes don't work, you're ready to start the construction phase.

In the cases where the design isn't complete, or the tests are fragmentary, it makes sense to remain in the elaboration phase for those classes, modules, or functions. Once the tests are understood, construction has a clear and achievable goal.

We don't always get the test cases right the first time, we must change them as we learn. We rarely get the working code right the first time. If the test cases come first, they make sure we have a clear goal.

In some cases, the design may not be easy to articulate without first writing some "spike solution" to explore an alternative. Once the spike works, it makes sense to write tests to demonstrate the code works.

See <http://www.extremeprogramming.org/rules/spike.html> for more on creating spike solutions.

At this point, you have an idea of how the software will be designed. The test cases are a way to formalize the design into a goal. It's time to begin construction.

1.2.4 Construction

The construction phase finishes the class and function (and module and package) definitions started in the elaboration phase. In some cases, test cases will need to be added as the definitions expand.

As we get closer to solving the problem, the number of tests passed will grow.

The number of tests may also grow. It's common to realize the sketch of a class definition is incomplete and requires additional classes to implement the **State** or **Strategy** design pattern. As another example, we may realize subclasses are required to handle special cases. This new understanding will change the test suite.

When we look at our progress over several days, we should see that the number of tests pass approaches the total number of tests.

How many tests do we need? There are strong opinions here. For the purposes of showing high-quality work, tests that exercise 100% of the code are a good starting point. For some industries, a more strict rule is to cover 100% of the logic paths through the code. This higher standard is often used for applications like robotics and health care where the consequences of a software failure may involve injury or death.

1.2.5 Transition

For enterprise applications, there is a transition from the development team to formal operations. This usually means a deployment into a production environment with the real user community and their data.

In organizations with good Continuous Integration/Continuous Deployment (CI/CD) practices, there will be a formalized execution of the `tox` command to make sure everything works: all the tests pass.

In some enterprises, the `make html` command will also be run to create the documentation.

Often, the technical operations team will need specific topics in the documentation and the `README.md` file. Operations staff may have to diagnose and troubleshoot problems with hundreds of applications, and they will need very specific advice in places where they can find it immediately. We won't emphasize this in this book, but as we complete our projects, it's important to think that our colleagues will be using this software, and we want their work life to be pleasant and productive.

The final step is to post your project to your public repository of choice.

You have completed part of your portfolio. You'll want potential business partners or hiring managers or investors to see this and recognize your level of skill.

We can view a project as a sequence of steps. We can also view a project as a deliverable set of files created by those steps. In the next section, we'll look over the deliverables in a little more detail.

1.3 List of deliverables

We'll take another look at the project, this time from the view of what files will be created. This will parallel the outline of the activities shown in the previous section.

The following outline shows many of the files in a completed project:

- The documentation in the `docs` directory. There will be other files in there, but you'll be focused on the following files:
 - The Sphinx `index.rst` starter file with references to overview and API sections.
 - An `overview.rst` section with a summary of the project.
 - An `api.rst` section with `.. automodule::` commands to pull in documentation from the application.
- A set of test cases in the `tests` directory.
 - Acceptance tests aimed at Behave (or the **pytest-bdd** plug-in for Gherkin). When using Behave, there will be two sub-directories: a

`features` directory and a `steps` directory. Additionally, there will be an `environment.py` file.

- Unit test modules written with the **pytest** framework. These all have a name that starts with `test_` to make them easy for **pytest** to find. Ideally, the **Coverage** tool is used to assure 100% of the code is exercised.
- The final code in the `src` directory. For some of the projects, a single module will be sufficient. Other projects will involve a few modules. (Developers familiar with Java or C++ often create too many modules here. The Python concept of *module* is more akin to the Java concept of *package*. It's not common Python practice to put each class definition into a separate module file.)
- Any JupyterLab notebooks can be in the `notebooks` folder. Not all projects use JupyterLab notebooks, so this folder can be omitted if there are no notebooks.
- A few other project files are in the top-level directory.
 - A `tox.ini` file should be used to run the **pytest** and **behave** test suites.
 - The `pyproject.toml` provides a number of pieces of information about the project. This includes a detailed list of packages and version numbers to be installed to run the project, as well as the packages required for development and testing. With this in place, the **tox** tool can then build virtual environments using the `requirements.txt` or the **pip-tools** tool to test the project. As a practical matter, this will also be used by other developers to create their working desktop environment.
 - An `environment.yml` can help other developers use **conda** to create their environment. This will repeat the contents of `requirements-dev.txt`. For a small team, it isn't helpful. In larger enterprise work groups, however, this can help others join your project.
 - Also, a `README.md` (or `README.rst`) with a summary is essential. In many cases, this is the first thing people look at; it needs to provide an "elevator pitch" for the project (see

<https://www.atlassian.com/team-playbook/plays/elevator-pitch>).

See <https://github.com/cmawer/reproducible-model> for additional advice on structuring complex projects.

We've presented the files in this order to encourage following an approach of writing documentation first. This is followed by creating test cases to assure the documentation will be satisfied by the programming.

We've looked at the development activities and a review of the products to be created. In the next section, we'll look at some suggested development tools.

1.4 Development tool installation

Many of the projects in this book are focused on data analysis. The tooling for data analysis is often easiest to install with the **conda** tool. This isn't a requirement, and readers familiar with the **PIP** tool will often be able to build their working environments without the help of the **conda** tool.

We suggest the following tools:

- **Conda** for installing and configuring each project's unique virtual environment.
- **Sphinx** for writing documentation.
- **Behave** for acceptance tests.
- **Pytest** for unit tests. The **pytest-cov** plug-in can help to compute test coverage.
- **Pip-Tool** for building a few working files from the `pyproject.toml` project definition.
- **Tox** for running the suite of tests.
- **Mypy** for static analysis of the type annotations.
- **Flake8** for static analysis of code, in general, to make sure it follows a consistent style.

One of the deliverables is the `pyproject.toml` file. This has all of the metadata about the project in a single place. It lists packages required by the application, as well as the tools used for development and testing. It helps to pin exact version numbers, making it easier for someone to rebuild the virtual environment.

Some Python tools — like PIP — work with files derived from the `pyproject.toml` file. The **pip-tools** creates these derived files from the source information in the TOML file.

For example, we might use the following output to extract the development tools information from `pyproject.toml` and write it to `requirements-dev.txt`.

```
% conda install -c conda-forge pip-tools
% pip-compile --extra=dev --output-file=requirements-dev.txt
```

It's common practice to then use the `requirements-dev.txt` to install packages like this:

```
% conda install --file requirements-dev.txt --channel=conda-forge
```

This will try to install all of the named packages, pulled from the community `conda-forge` channel.

Another alternative is to use PIP like this:

```
% python -m pip install --r requirements-dev.txt
```

This environment preparation is an essential ingredient in each project's inception phase. This means the `pyproject.toml` is often the first deliverable created. From this, the `requirements-dev.txt` is extracted to build environments.

To make the preceding steps and deliverables more specific, we'll walk through an initial project. This project will help show how the remaining projects should be completed.

1.5 Project 0 – Hello World with test cases

This is our first project. This project will demonstrate the pattern for all of the book's projects. It will include these three elements.

- **Description:** The description section will set out a problem, and why a user needs software to solve it. In some projects, the description will have very specific details. Other projects will require more imagination to create a solution.
- **Approach:** The approach section will offer some guidance on architectural and design choices. For some projects there are trade-offs, and an **Extras** section will explore some of the other choices.
- **Deliverables:** The deliverables section lists the expectations for the final application or module. It will often provide a few Gherkin feature definitions.

For this initial project, the description isn't going to be very complicated. Similarly, the approach part of this first project will be brief. We'll dwell on the deliverables with some additional technical discussion.

1.5.1 Description

The problem the users need to solve is how best to bring new developers on board. A good onboarding process helps our users by making new members of the team as productive as quickly as possible. Additionally, a project like this can be used for experienced members to introduce them to new tools.

We need to guide our team members in installing the core set of development tools, creating a working module, and then displaying their completed work at the end of a sprint. This first project will use the

most important tools and assure that everyone has a common understanding of the tools and the deliverables.

Each developer will build a project to create a small application. This application will have a command-line interface (CLI) to write a cheerful greeting.

The expectations are shown in the following example:

```
% python src/hello_world.py --who "World"  
Hello, World!
```

This example shows how running the application with a command-line parameter of `--who "world"` produces a response on the console.

1.5.2 Approach

For this project, the objective is to create a Python application module. The module will need several internal functions. The functions can be combined into a class, if that seems more appropriate. The functions are these:

- A function to parse the command-line options. This will use the `argparse` module. The default command-line argument values are available in `sys.argv`.
- A function to write a cheerful greeting. This is, perhaps, only a single line of code.
- An overall function with an obvious name like `main()` to get the options and write the greeting.

The module, as a whole, will have the function (or class) definitions. It will also have an `if __name__ == "__main__":` block. This block will guard the evaluation of the expression `main()` to make the module easier to unit test.

This is quite a bit of engineering for a simple problem. Some might call it over-engineering. The idea is to create something with enough complexity that more than one unit test case is required.

1.5.3 Deliverables

As noted above in *[List of deliverables](#)*, there are a number of deliverable files for projects in general. Here are the suggested files for this project:

- `README.md` summarizes the project.
- `pyproject.toml` defines the project, including development tools, test tools, and other dependencies.
- `docs` contains the documentation. As described above, this should be built by the `sphinx-quickstart` tool and should contain at least an overview and an API section.
- `tests` contains test cases; the files include the following:
 - `test_hw.py` contains unit tests for the module's functions or classes.
 - `features/hello_world.feature` has an overall acceptance test as a collection of scenarios.
 - `steps/hw_cli.py` has Python definitions for the steps in the scenarios.
 - `environment.py` contains functions to control **behave**'s test setup and teardown. For simple projects, it may be empty.
- `tox.ini` configuration for the **tox** tool to run the complete test suite.
- `src` contains the `hello_world.py` module.

We'll look at a few of these files in detail in the following sub-sections.

The `pyproject.toml` project file

The `pyproject.toml` file contains a great deal of project metadata in a single location. The minimal content of this file is a description of the "build-system" used to build and install the package.

For the purposes of this book, we can use the following two lines to define the build system:

```
[build-system]
requires = ["setuptools", "wheel"] # PEP 508 specifications.
```

This specifies the use of the `setuptools` module to create a “wheel” with the project’s code. The `pyproject.toml` doesn’t need to define the distribution package in any more detail. This book doesn’t emphasize the creation of a distribution package or the management of packages with the Python Package Index, PyPI.

The rest of this file should have information about the project. You can include a section like the following:

```
[project]
name = "project_0"
version = "1.0.0"
authors = [
    {name = "Author", email = "author@email.com"},
]
description = "Real-World Python Projects -- Project 0."
readme = "README.md"
requires-python = ">=3.10"
```

Clearly, you’ll want to update the `authors` section with your information. You may be using a newer version of Python and may need to change the `requires-python` string to specify the minimum version required for your unique solution.

The `[project]` section needs three other pieces of information:

- The packages required to execute your application.
- Any packages or tools required to test your application.
- Any packages or tools required to develop your application.

These three dependencies are organized as follows:

```
dependencies = [  
    # Packages required -- None for Project Zero.  
]  
[project.optional-dependencies]  
dev = [  
    # Development tools to work on this project  
    "sphinx==7.0.1",  
    "sphinxcontrib-plantuml==0.25",  
    "pip-tools==6.13.0"  
]  
test = [  
    # Testing tools to test this project  
    "pytest==7.2.0",  
    "tox==4.0.8",  
    "behave==1.2.6"  
]
```

The `dependencies` line lists the dependencies required to execute the application. Some projects — like this one — rely on the standard library, and nothing more needs to be added. The `[project.optional-dependencies]` section contains two lists of additional packages: those required for development, and those required for testing.

Note that we’ve put specific version numbers in this file so that we can be absolutely certain what packages will be used. As these packages evolve, we’ll need to test newer versions and upgrade the dependencies.

If you see the version numbers in this book are behind the current state of the art on PyPI or Conda-Forge, feel free to use up-to-date versions.

It helps to use the **pip-compile** command. This is installed as part of **pip-tools**. This command create extract files from the `pyproject.toml` file for use by **pip** or **conda**.

For developers, we often want to install all of the “extras.” This usually means executing the following command to create a `requirements-dev.txt` file that can be used to build a development environment.


```
% pip-compile --all-extras -o requirements-dev.txt
```

In order to run the **tox** tool, it's common to also create a testing-only subset of the required packages and tools. Use the following command:

```
% pip-compile --extra test -o requirements.txt
```

This creates the `requirements.txt` to be used to detect manage virtual environments used by **tox** for testing.

The docs directory

As noted above in ***Suggested project sprints*** this directory should be built with `sphinx-quickstart`. After the initial set of files is created, make the following changes:

- Add a `api.rst` file as a placeholder for the Sphinx-generated API documentation. This will use the `.. automodule::` directive to extract documentation from your application.
- Add a `overview.rst` file with an overview of the project.
- Update the `index.rst` to include these two new files in the table of contents.
- Update the `conf.py` to append the `src` directory to `sys.path`. Also, the `sphinx.ext.autodoc` extension needs to be added to the `extensions` setting in this file.

The `make html` command in the `docs` directory can be used to build the documentation.

The tests/features/hello_world.feature file

The `features` directory will have Gherkin-language definitions of the features. Each feature file will contain one or more scenarios. For larger projects, these files often start with statements lifted from problem de-

scriptions or architectural overviews that are later refined into more detailed steps to describe an application's behavior.

For this project, one of the feature files should be `features/hello_world.feature`. The contents of this file should include a description of the feature and at least one scenario. It would look like the following example:

```
Feature: The Cheerful Greeting CLI interface provides a greeting
        to a specific name.
```

```
Scenario: When requested, the application writes the greeting message.
          When we run command "python src/hello_world.py"
          Then output has "Hello, World!"
```

There's no `Given` step in this scenario; there's no initialization or preparation required. Each of the steps has only a single clause, so there are no `And` steps, either.

This example doesn't precisely match the example in the description. There are two possible reasons for this: one of the two examples is wrong, or, more charitably, this example hints at a second feature.

The idea implied by this example is there's a default behavior when no `--who` command-line option is provided. This suggests that a second scenario — one with the `--who` option should be added for this feature.

The tests/steps/hw_cli.py module

The `steps` directory contains modules that define the natural-language phrases in the feature files. In the `hello_world.feature` file the `When` and `Then` steps had phrases written out in plain English:

- We run the command "python src/hello_world.py"
- Output has "Hello, World!"

The `steps/hw_cli.py` module will map the step's phrases to Python functions. It works by using decorators and pattern-matching to specify the type of step (`@given`, `@when`, or `@then`) and the text to match. The presence of `{parameter}` in the text will match the text and provide the value matched to the step function as an argument. The function names are irrelevant and are often `step_impl()`.

Generally, the `@given` steps will accumulate parameter values in the test context object. Best practices suggest there should be only one `@when` step; this will perform the required operation. For this project, it will run the application and gather the output files. The `@then` steps can use `assert` statements to compare actual results against the expected results shown in the feature file.

Here's how the `steps/hw_cli.py` module might look:

```
import subprocess
import shlex
from pathlib import Path

@when(u'we run command "{command}"')
def step_impl(context, command):
    output_path = Path("output.log")
    with output_path.open('w') as target:
        status = subprocess.run(
            shlex.split(command),
            check=True, text=True, stdout=target, stderr=subprocess.STDOUT)
    context.status = status
    context.output = output_path.read_text()
    output_path.unlink()

@then(u'output has "{expected_output}"')
def step_impl(context, expected_output):
    assert context.status.returncode == 0
    assert expected_output in context.output
```

This assumes a relatively small output file that can be collected in memory. For a larger file, it would make sense for the `@when` step to create a

temporary file and save the file object in the context. The `@then` step can read and close this file. The `tempfile` module is handy for creating files that will be deleted when closed.

An alternative is to create a `Path` object and save this object in the context. The `@when` step can write output to this path. The `@then` step can read and examine the contents of the file named by the `Path` object.

When a test step detects a problem with an `assert` statement, it may not finish completely. The approach of using a `Path` object requires some care to be sure the temporary files are deleted. The `environment.py` module can define an `after_scenario(context, scenario)` function to remove temporary files.

The tests/environment.py file

This module will contain some function definitions used by **behave**. For this project, it will be empty. The module must be present; a module docstring is appropriate to explain that it's empty.

The `tests/steps` module for this example will have examples that can be refactored into two potentially reusable functions for executing an application and checking the output from an application for specific text. This additional design effort isn't part of this project. You may find it helpful to do this refactoring after completing several of these projects.

Once the features, steps, and environment are in place, the **behave** program can be used to test the application. If there's no application module, the tests will fail. Creating a skeleton application module in the `src` directory will allow the test case to execute and fail because the output isn't what was expected.

The tests/test_hw.py unit tests

A unit test can be implemented as a **pytest** function that uses a fixture, `capsys`, to capture the system output. The unit test case expects the application to have a `main()` function that parses the command-line options.

Here's one suggested unit test function:

```
import hello_world

def test_hw(capsys):
    hello_world.main([])
    out, err = capsys.readouterr()
    assert "Hello, World!" in out
```

Note the test for the `main()` function provides an explicit empty list of argument values. It is essential to override any value for `sys.argv` that might be present when **pytest** is running.

The `hello_world` module is imported by this test. There are two important consequences of this import:

- The `src/hello_world.py` module must have an `if __name__ == "__main__":` section. A simple Python script (without this section) will execute completely when imported. This can make testing difficult.
- The `src` directory must be part of the `PYTHONPATH` environment variable. This is handled by the `tox.ini` file.

This test will tolerate additional output in addition to the required cheerful greeting. It might make sense to use something like `"Hello, World!" == out.strip()`.

The implementation details of the `main()` function are opaque to this test. This `main()` function could create an instance of a class; it could use a static method of a class, also.

The `src/tox.ini` file

Now that the tests exist, we can run them. The **tox** (and **nox**) tools are ideal for running a suite of tests.

Here's an example `tox.ini` file:

```
[tox]
min_version = 4.0
skipdist = true

[testenv]
deps = pip-tools
      pytest
      behave
commands_pre = pip-sync requirements.txt
setenv =
    PYTHONPATH=src
commands =
    pytest tests
    behave tests
```

This file lists the tools used for testing: **pip-tools**, **pytest**, and **behave**. It provides the setting for the `PYTHONPATH`. The `commands_pre` will prepare the the virtual environment using the **pip-sync** command that is part of the **pip-tools** package. The given sequence of commands defines the test suite.

The `src/hello_world.py` file

This is the desired application module. The test framework is helpful to confirm that it really does work, and — more importantly — it meets the definition of done provided in the `*.feature` files.

As we noted above, the unit tests will import this app as a module. The acceptance test, in contrast, will run the app. This means the `if __name__ == "__main__":` section is essential.

For a small application like this, the real work of the application should be encapsulated in a `main()` function. This allows the main module to

end with the following snippet:

```
if __name__ == "__main__":  
    main()
```

This assures that the module will not take off and start running when imported. It will only do useful work when invoked from the command line.

1.5.4 Definition of done

This project is tested by running the `tox` command.

When all of the tests execute, the output will look like this:

```
(projectbook) slott@MacBookPro-SLott project_0 % tox  
py: commands[0]> pytest tests  
...  
py: commands[1]> behave tests  
...  
py: OK (0.96=setup[0.13]+cmd[0.53,0.30] seconds)  
congratulations :) (1.55 seconds)
```

This output has elided the details from **pytest** and **behave**. The output from the **tox** tool is the important summary `py: OK`. This tells us all the tests passed.

Once this is complete, we can run the following to create the API documentation:

```
% (cd docs; make html)
```

It can help to wrap the two commands with `()` so the `cd docs` command doesn't leave the console session in the `docs` directory. Some developers prefer to have two windows open: one in the top-level direc-

tory to run the **tox** tool and one in the **docs** subdirectory to run the **make** commands for the **sphinx** tool.

1.6 Summary

In this chapter, we've looked at the following topics:

- An overview of the software quality principles that we'll try to emphasize.
- A suggested approach to completing the project as a sequence of project sprints.
- A general overview of the list of deliverables for each project.
- The tools suggested for creating these examples.
- A sample project to act as a template for subsequent projects.

After creating this initial project, the next chapter will look at the general collection of projects. The idea is to create a complete data analysis tool set with a number of closely-related projects.

1.7 Extras

Here are some ideas for you to add to this project.

1.7.1 Static analysis - mypy, flake8

There are several common static analysis tools that are as essential as automated testing:

- **mypy** checks type annotations to be sure the functions and classes will interact properly.
- **flake8** does other syntax checks to make sure the code avoids some of the more common Python mistakes.
- **black** can be used to check the formatting to make sure it follows the recommended style. The **black** application can also be used to re-format a new file.

- **isort** can be used to put a long collection of `import` statements into a consistent order.

Once the application passes the functional tests in the `*.feature` files, these additional non-functional tests can be applied. These additional tests are often helpful for spotting more nuanced problems that can make a program difficult to adapt or maintain.

1.7.2 CLI features

The command-language interface permits a single option, the `--who` option, to provide a name.

It makes sense to add a scenario to exercise this option.

What should happen with the `--who` is provided without a value? Is the following appropriate?

```
(projectbook) slott@MacBookPro-Slott project_0 % python src/hello_world.py
--who

usage: hello_world.py [-h] [--who WHO]
hello_world.py: error: argument --who/-w: expected one argument
```

Should the help be extended to clarify what's required?

Consider adding the following scenarios (and the implementing code):

- Add a scenario for the `--help` option, which is provided automatically by the `argparse` module.
- Add a scenario for the `--who` with no value error.

1.7.3 Logging

Consider a more complex application where additional debugging output might be helpful. For this, it's common to add a `--verbose` option

to set the logging level to `logging.DEBUG` instead of a default level of `logging.INFO`.

Adding this option requires adding logging capabilities. Consider making the following changes to this module:

- Import the `logging` module and create a global logger for the application.
- Update the `main()` function to set the logger's level based on the options.
- Update the `__name__ == "__main__":` block to have two lines: `logging.basicConfig()` and `main()`. It's best to keep logging configuration isolated from the rest of the application processing.

1.7.4 Cookiecutter

The `cookiecutter` project (see <https://cookiecutter.readthedocs.io/en/stable/>) is a way to build a template project. This can help team members get started by sharing a single template. As tool versions or solution architectures change, additional cookie-cutter templates can be developed and used.

There are thousands of cookie-cutter templates available. It can be difficult to locate one that's suitably simple. It may be better to create your own and add to it as new concepts are introduced in later chapters.